

Guia de Automação de Emails de Boas-vindas

Visão Geral

O sistema de emails está configurado com 3 tipos de notificação:

1. **Email de Boas-vindas** (Imediato) - Enviado no signup
2. **Email Dia 3** - Enviado 3 dias após cadastro
3. **Email Dia 7** - Enviado 7 dias após cadastro

Endpoints Disponíveis

1. Email de Boas-vindas (Enviado Automaticamente)

Endpoint: POST /api/email/send-welcome

Chamado automaticamente quando novo usuário se cadastra via /api/signup.

Payload:

```
{
  "userId": "user-id-uuid",
  "userEmail": "usuario@example.com",
  "userName": "Seu Nome"
}
```

Resposta:

```
{
  "success": true,
  "message": "Welcome email sent"
}
```

2. Email Dia 3 (Primeiros Passos)

Endpoint: POST /api/email/send-day3

Deve ser chamado 3 dias após o cadastro do usuário.

Payload:

```
{
  "userId": "user-id-uuid",
  "userEmail": "usuario@example.com",
  "userName": "Seu Nome"
}
```

Conteúdo:

- Parabéns pelos primeiros 3 dias

- Tarefas recomendadas (categorias, primeira receita, teste de venda)
 - Dica do dia sobre recursos
-

3. Email Dia 7 (Próximo Passo)

Endpoint: `POST /api/email/send-day7`

Deve ser chamado 7 dias após o cadastro do usuário.

Payload:

```
{
  "userId": "user-id-uuid",
  "userEmail": "usuario@example.com",
  "userName": "Seu Nome"
}
```

Conteúdo:

- Parabéns pela primeira semana
 - Benefícios do upgrade (transações ilimitadas, relatórios, IA)
 - CTA para página de pricing
-

Configuração de Automação

Opção 1: Usar Daemon/Scheduled Task (Recomendado)

Use a ferramenta `perform_subtask_daemon_management` para criar tarefas agendadas que enviem os emails automaticamente.

Exemplo para Email Dia 3:

```
Criar daemon task que:
1. Busca todos os usuários criados há exatamente 3 dias
2. Para cada usuário, faz POST para /api/email/send-day3
3. Executa diariamente às 8:00 (antes dos negócios abrirem)
```

Exemplo para Email Dia 7:

```
Criar daemon task que:
1. Busca todos os usuários criados há exatamente 7 dias
2. Para cada usuário, faz POST para /api/email/send-day7
3. Executa diariamente às 8:00
```

Opção 2: Usar Cron Job Linux

```
# Email Dia 3 - Execute todos os dias às 8:00 AM
0 8 * * * curl -X POST https://seu-app.com/api/email/send-day3-batch \
-H "Content-Type: application/json" \
-d '{"batchProcess": true}'

# Email Dia 7 - Execute todos os dias às 8:00 AM
0 8 * * * curl -X POST https://seu-app.com/api/email/send-day7-batch \
-H "Content-Type: application/json" \
-d '{"batchProcess": true}'
```

Opção 3: Criar Endpoints em Batch

Para automatizar melhor, você pode criar endpoints `/api/email/send-day3-batch` e `/api/email/send-day7-batch` que:

1. Consultam o banco de dados para encontrar usuários “elegíveis”
2. Chamam os endpoints individuais para cada usuário
3. Retornam estatísticas de sucesso/erro

Pseudo-código:

```
// /api/email/send-day3-batch/route.ts
export async function POST() {
  // 1. Find users created exactly 3 days ago
  const threeDaysAgo = new Date(Date.now() - 3 * 24 * 60 * 60 * 1000);
  const users = await prisma.user.findMany({
    where: {
      createdAt: {
        gte: new Date(threeDaysAgo.setHours(0, 0, 0, 0)),
        lt: new Date(threeDaysAgo.setHours(24, 0, 0, 0)),
      },
      emailSentDay3: false, // Track if already sent
    },
  });

  // 2. Send email to each user
  for (const user of users) {
    await fetch('/api/email/send-day3', {
      method: 'POST',
      body: JSON.stringify({
        userId: user.id,
        userEmail: user.email,
        userName: user.name,
      }),
    });

    // Mark as sent
    await prisma.user.update({
      where: { id: user.id },
      data: { emailSentDay3: true },
    });
  }

  return Response.json({ sent: users.length });
}
```

Variáveis de Ambiente

As notificações já foram registradas e configuradas:

```
NOTIF_ID_EMAIL_DE_BOASVINDAS=2efc95ae0
NOTIF_ID_EMAIL_PRIMEIROS_PASSOS_DIA_3=9f7f7c221
NOTIF_ID_EMAIL_PRXIMO_PASSO_DIA_7=1100262962
```

Monitoramento e Logs

Todos os emails são enviados através da API da Abacus.AI:

```
https://apps.abacus.ai/api/sendNotificationEmail
```

Se algo der errado:

1. Verifique se `ABACUSAI_API_KEY` está configurado
2. Verifique se `WEB_APP_ID` está configurado
3. Consulte os logs da aplicação para erros de envio

Template dos Emails

Email 1: Boas-vindas

- **Assunto:** “Bem-vindo ao Restaurantes! 🍽️”
- **Cor:** Azul/Roxo (gradiente)
- **CTA:** Ir para Dashboard
- **Dica:** “Em 3 dias você receberá dicas especiais”

Email 2: Dia 3

- **Assunto:** “[Nome], confira suas primeiras dicas 📖”
- **Cor:** Verde/Ciano (gradiente)
- **CTA:** Ver Guia Completo
- **Conteúdo:** Tarefas recomendadas para os próximos passos

Email 3: Dia 7

- **Assunto:** “[Nome], está pronto para o próximo nível? 🚀”
- **Cor:** Roxo/Rosa (gradiente)
- **CTA:** Ver Planos de Upgrade
- **Conteúdo:** Benefícios do upgrade e incentivo

Próximos Passos

1. **Criar campo no banco** (opcional):

```
sql
```

```
ALTER TABLE "User" ADD COLUMN "emailSentDay3" BOOLEAN DEFAULT false;
```

```
ALTER TABLE "User" ADD COLUMN "emailSentDay7" BOOLEAN DEFAULT false;
```

2. **Criar endpoints em batch** para processar múltiplos usuários

3. **Agendar tasks** usando `perform_subtask_daemon_management`

4. **Testar manualmente:**

```
bash
```

```
# Test welcome email
```

```
curl -X POST http://localhost:3000/api/email/send-welcome \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "userId": "test-user-id",
```

```
  "userEmail": "seu-email@example.com",
```

```
  "userName": "Seu Nome"
```

```
}'
```

Notas Importantes

- Os emails são enviados com o formato HTML para melhor visual
- Cada email tem branding personalizado (sender_alias = nome do app)
- Os usuários podem desabilitar estas notificações nas configurações
- Se uma notificação for desabilitada, a API retorna `notification_disabled: true`